1. Навіщо в програмуванні окремий тип для «так» і «ні»?

Це потрібно для зручності та надійності. Тип boolean має лише два значення: true або false. Замість того, щоб запам'ятовувати, що "1" — це так, а "0" — це ні, ми використовуємо спеціальний тип. Це робить код більш зрозумілим, схожим на звичайну мову, і допомагає уникнути дурних помилок, коли замість "так/ні" випадково підставляють якесь число.

2. Як сказати !(a && b) людською мовою?

Якщо перекласти це простою мовою, то це означає: **«Неправда, що виконані обидві умови одночасно»**. Іншими словами, це означає, що хоча б одна з умов (або навіть обидві) не виконана.

3. Чим && відрізняється від &?

Обидва знаки означають логічне "І", але працюють по-різному:

- **&& (подвійний амперсанд):** Це "розумне" І. Якщо він перевіряє першу умову і бачить, що вона вже false (хибна), він навіть не буде дивитися на другу умову, бо результат і так уже зрозумілий. Це економить час комп'ютера.
- **& (одинарний амперсанд):** Це "вперте" І. Він завжди чесно перевірятиме обидві умови від початку до кінця, незалежно від того, що показала перша умова.

4. Чому програма не падає в if (x != null && x.name == "Іван"), хоча x порожній?

Вся магія в операторі &&. Комп'ютер починає читати зліва направо. Він бачить першу перевірку: x != null (чи існує x?). Оскільки x порожній, ця перевірка дає false (ні). Завдяки &&, комп'ютер розуміє: "Ага, якщо перше — брехня, то і весь вираз — брехня". Він одразу зупиняється і навіть не намагається прочитати x.name == "Іван". Якби він спробував прочитати ім'я у порожнього об'єкта, програма б точно впала з помилкою (NullPointerException), але && нас врятував.

5. Як запхати 32 відповіді «так/ні» в одне число?

Звичайне ціле число (тип int) всередині комп'ютера зберігається як рядок із 32 нулів та одиниць (бітів). Ми можемо домовитися, що кожен з цих бітів — це окреме "так" (1) або "ні" (0). Таким чином, замість того, щоб створювати 32 окремі змінні типу boolean, ми беремо одне число int, і кожен його біт відповідатиме на одне з 32 запитань.

6. Що швидше: n % 2 чи n & 1?

Швидшим буде варіант **n & 1**. Для комп'ютера побітові операції (такі як &) є рідними, вони виконуються на найнижчому рівні процесора майже миттєво (за один такт). Операція остачі від ділення (%) — це складніша математична дія, яка вимагає більше часу. Тому, коли комп'ютер просто перевіряє останній біт числа через & 1, щоб дізнатися, чи воно парне/непарне, це працює набагато швидше.